Relational databases
Databases Normalisation

Nicolae-Cosmin Vârlan

Relationa model - Keys (from the first course/laboratory)

- Super key - an attribute or a set of attributes that uniquely identify each touple in a relation;
- Candidate key - a super key with the property that it does not contain a subset of attributes that are a key;
- Primary key - a candidate key selected to represent each row in the relation;
- Alternate key - any other candidate key except the one selected as primary key;
- Foreign key - an attribute or a set of attributes from a relation which refer some candidate key from other table.

Finding candidate keys using functional dependencies

Observation: The candidate key is, in fact, formed from a combination of attributes $X \in U$ which can uniquely identify each touple (line). If they can uniquely identify the line (meaning the value from all the attributes), then we can consider that there is functional dependency from X to the rest of attributes $(U - X)$.

Formaly: Let $R[U]$ be a relation schema and Σ a set of functional dependencies satisfied by $R[U]$. $X \subseteq U$ is a candidate key iff $X^+ = U$ and $\forall X' \subseteq X, X'^+ \neq U$

An attribute is called key-attribute (or prime) if it belongs to at least one candidate key.

An attribute is non-key-attribute (or not prime if it doesn't belong to any candidate key.

Example 1:

Consider $U = \{A, B, C\}$ and $\Sigma = \{A \to B, B \to C\}$. We will cosnider the set $X^+_{\mathcal{R}_A}$ for each attribute:

$A^+_{\mathcal{R}_A} = \{A, B, C\}$	(A obtained from Armstrong's axiom A_1 B from $A \to B$, C from $A \to B$, $B \to C$ by using A_3 A_3)
$B^+_{\mathcal{R}_A} = \{B, C\}$	(B from A_1, C from $B \to C$)
$C^+_{\mathcal{R}_A} = \{C\}$	(C from A_1)

In this case, A is the candidate key (CK) because $A^+ = U$.

Prime attributes (key-attributes): $\{A\}$
Non-prime attributes (non key-attributes): $\{B, C\}$

Consider $U = \{A, B, C\}$ and $\Sigma = \{A \to B, B \to C\}$ (same example). A different method for finding the candidate key (CK) is:

Organize the attributes in 3 sets: Left, Middle, Right depending where the attribute is in the functional dependencies in Σ:

- Left: Attribute appears only in left-sides of the dependencies in Σ or that are not in any dependencies.
- Middle: Attribute appear in both left and right sides in dependencies in Σ.
- Right: Attribute appears only in right-sides of the dependencies in Σ.

Left	Middle	Right
A	B	C

Rule: Left attributes are key attributes (and belong to all candidate keys).
Rule: Right attributes are non-key-attributes (not prime).

Example 2:

Fie $U = \{A, B, C, D, E, F\}$ și
$\Sigma = \{A \to BD, B \to C, DE \to F\}$.
What are the candidate keys ?

Left	Middle	Right
A, E	B, D	C, F

$A^+_{\mathcal{R}_A} = \{A, B, C, D\}$ - is not CK (A^+ does not contain F).

Actually, we established that all attributes in the Left set belong to any CK. In our case, both A and E should belong the any CK:
$AE^+_{\mathcal{R}_A} = \{A, B, C, D, E, F\}$

$AE =$ multivalued key (it is composed from more than one attribute).

Example 3:

Let $U = \{A, B\}$ and $\Sigma = \{A \to B, B \to A\}$. CK ?

Left	Middle	Right
	A, B	

$A^+_{\mathcal{R}_A} = \{A, B\}$
$B^+_{\mathcal{R}_A} = \{A, B\}$

Both A and B are CK.

Prime attributes (key attributes): $\{A, B\}$
Non-prime attributes: $\emptyset$

Full dependencies

Let $R[U]$ be a relationa schema over a set of attributes U and Σ a set of functional dependencies that are satisfied by $R[U]$. A dependency $X \to A \in \Sigma^+$ is a full dependency if $\nexists X' \subset X$ such that $X' \to A \in \Sigma^+$.

Example:
$R[A, B, C, D]$
$\Sigma = \{AB \to C, B \to D, BC \to A\}$

All dependencies Σ are full (e.g. Σ^+ doesn't contain any of the dependencies: $A \to C$, $B \to C$, $B \to A$, $C \to A$).

$AB \to D$ is not full because $B \subset AB$ and we have $B \to D \in \Sigma^+$

Transitive dependent attribute

Let $R[U]$ be a relational schema built over the set o attributes U and Σ a set of functional dependencies that are satisfied by $R[U]$. An attribute $A \in U$ is transitive dependent of X $(X \subset U, A \notin X)$ if $\exists Y \subset U$ s.t.:

- $A \in U - Y$
- $X \to Y \in \Sigma^+$
- $Y \to A \in \Sigma^+$
- $Y \to X \notin \Sigma^+$

Example:
$R[A, B, C, D, E]$
$\Sigma = \{AB \to C, AB \to D, CD \to E\}$

E is transitive dependent of AB $(X = AB, Y = CD)$.

Trivial dependencies

A functional dependency $X \rightarrow Y$ is trivial iff $Y \subseteq X$

A multivalued dependency $X \twoheadrightarrow Y$ is trivial if $Y \subseteq X$ or if $X \cup Y = U$

Database Normalisation

There are two reasons for normalizing a database:

- Removing redundancies.
- To always have consistent data (no errors by duplication).

Normalization has to be done before entering any data in the database. That is why we will talk about SCHEMA NORMALIZATION and not about normalizing some relations.

There are different (classic) normal forms, each one having more restrictions than the ones before: 1NF, 2NF, 3NF, BCNF, 4NF.

Let's see them...

1NF (1971)

Let U be a set of attributes and $R[U]$ a relation schema.
We will say that $R[U]$ is in first normal form (1NF) iff the domain of each attribute in U containis only atomic values.

If the schema is not in 1NF, we can apply the following steps to normalize it:

- Remove the non-atomic atributes from the schema.
- Identify a key in each row (where the non-atomic attribute was).
- Create a new table where the key can be associated with each element from the non-atomic attribute.

1NF

An example could be with some persons having more then one first name. In this case, a secondary table can be created containing, on each touple, the ID of the person together with one of its first names.
We do not alter the relation because we could tell that there might be the case that a person has more then one first name. So this step can be done even before entering the data in the tables.
However, there might be a problem. The ID of the person could be seen as a key. However, in this normal form we do not have associated any functional dependencies and there is no need for functional dependencies to see that an attribute has its domain on a carthesian product of atomic values.

2NF (1971)

A relation schema $R[U]$ which is in 1NF, together with a set of functional dependencies Σ is in the second normal form (2NF) if any non-prime attribute from $R[U]$ is fully dependent of any key of $R[U]$.

Continuing the example from full dependencies:
$R[A, B, C, D]$
$\Sigma = \{AB \rightarrow C, B \rightarrow D, BC \rightarrow A\}$

1. Find the candidate keys.
2. Find the non-prime attributes
3. Are the non-primes fully dependent from the keys we found in step 2 ?

2NF

Solve:
$R[A, B, C, D]$
$\Sigma = \{AB \to C, B \to D, BC \to A\}$

Candidate keys: AB, BC
Prime attributes: A, B, C
Non-prime attributes: D
$B \to D \in \Sigma^+$. D is not fully dependent from AB because we have both $AB \to D \in \Sigma^+$ and $B \to D \in \Sigma^+$ which mean that $R[U]$ is not in 2NF.

Observation: if we don't have multivalued keys, then $R[U]$ is in 2NF (because there are no subsets of a set formed with a single attribute - except the empty set and the set containing the attribute - and those are not important in the full dependency study).

2NF

Having a relational schema $R[U]$ which is not in 2NF, we can transform it in order to get it in 2NF following the next steps:

step 1: Identify candidate keys.
step 2: Find non-prime attributes.
step 3: For each non-prime attribute A, identify which are the attributes from the key that determine A.
step 4: Create a new relation R' over those attributes identified in the previous step together with the non-prime attribute for whom the non-prime dependency was found and eliminate the attribute A from the initial $R[U]$.

2NF

Example: If we have the folowing relational schema OS[name, version, year, company]

name	version	year	company
Windows	XP	2001	Microsoft
MacOS	Sierra	2017	Apple
Ubuntu	Bionic Beaver	2018	Ubuntu
Windows	7	2009	Microsoft
MacOS	Mojave	2018	Apple

We usually say that Windows XP is made by Microsoft. We have the dependency: name,version $\rightarrow$ company. Î n the same time, we also know that Windows is only made by Microsoft as is MacOS only made by Apple. So we have that name $\rightarrow$ company. Each version is made in a certain year. We have: version $\rightarrow$ year.

2NF

dependencies:

- name,version $\rightarrow$ company
- name $\rightarrow$ company
- version $\rightarrow$ year

Key: (name, version) (in the left of dependencies)
Non-prime attributes: company

company is not fully dependent because we have both name,version $\rightarrow$ company, and we also have that name $\rightarrow$ company. We will eliminate from our schema the attribute company and build a new relation R'[name, company].

We have that...

2NF

name	version	year
Windows	XP	2001
MacOS	Sierra	2017
Ubuntu	Bionic Beaver	2018
Windows	7	2009
MacOS	Mojave	2018

and

name	company
Windows	Microsoft
MacOS	Apple
Ubuntu	Ubuntu

Is that in 2NF ? We also have to check if year is fully dependent from the keys. You do that...

3NF (1971)

A relational schema $R[U]$ together with a set of functional dependencies Σ is int the 3rd normal form (3NF) if it is in 2NF and any non-key attribute is **NOT** transitive dependent from any key of R.

Example:
Consider the relational schema $R[A, B, C]$ and
$\Sigma = \{AB \rightarrow C, C \rightarrow A\}$

Keys: AB, BC
Non-keys attributes: $\emptyset$ - so it is in both 2NF and 3NF.

3NF - some joke about 3NF....

An approximation of Codd's definition of 3NF, paralleling the traditional pledge to give true evidence in a court of law, was given by Bill Kent: *[Every] non-key [attribute] must provide a fact about the key, the whole key, and nothing but the key.* [1]

A common variation supplements this definition with the oath: *so help me Codd* [2].

[1] Kent, William - A Simple Guide to Five Normal Forms in Relational Database Theory, Communications of the ACM 26 (2), Feb. 1983, pp. 120 – 125.

[2] Diehr, George. Database Management (Scott, Foresman, 1989), p. 331.

3NF

Normalisation example 3NF
Consider the schema: $olympiad[subject, year, winner, IQ]$.
We have the following functional dependencies:
$\Sigma = \{(subject, year) \rightarrow winner, winner \rightarrow IQ\}$

Key: $(materie, an)$
non-key attributes: $castigator, IQ$ (IQ is transitiv dependent)

We can observe that $(subject, year) \rightarrow IQ \in \Sigma^+$, and in the same time $subject \rightarrow IQ \notin \Sigma^+$ and $year \rightarrow IQ \notin \Sigma^+$. The relational schema is in 2NF (non-key attributes are fully dependent from the key).

3NF

Normalisation example 3NF
Consider the schema: $olympiad[subject, year, winner, IQ]$.
We have the following functional dependencies:
$\Sigma = \{(subject, year) \rightarrow winner, winner \rightarrow IQ\}$

Key: $(subject, year)$
non-key attributes: $winner, IQ$ (IQ is transitive dependent)

We can observe that IQ is transitive dependent from the key via then attribute $winner$ which is not a key.

Normalising $olympiad[subject, year, winner, IQ]$ we will get two different relations: $olympiad[subject, year, winner]$ and $olympiad[winner, IQ]$.

BCNF (Boyce-Codd Normal Form) $\equiv$ 3.5NF (1975)

A relational schema $R[U]$ together with a set of functional dependencies Σ is int the $Boyce - Codd$ normal form (BCNF) if it is in 1NF and for any non-trivial functional dependence $X \to A \in \Sigma^+$, X is a superkey in $R[U]$.

Observation: A Schema that is in BCNF is also in 3NF.

Observation: A Schema that is in BCNF is also in 3NF.

a) Suppose that (R, Σ) is in BCNF - therefore, by definition, is in 1NF.

b) Proof by contradiction: Suppose that (R, Σ) is not in 2NF, it means that there exists an non-key attribute A and a candidate key K such that $K \to A$ is not a full dependency. That mean that there exists $X \subsetneq K$ s.t. $X \to A \in \Sigma^+$.

The definition for BCNF stated that if $X \to A \in \Sigma^+$ then X is a superkey in $R[U]$.

So we have that $X \subsetneq K$ and that X is a superkey. That mean that K is not a candidate key so our supposition was false. Therefore (R, Σ) is in 2NF

Observation: A Schema that is in BCNF is also in 3NF.

c) Proof by contradiction: Suppose that (R, Σ) is not in 3NF (as we have seen, is for sure in 1NF, and in 2NF - from previous slide)

Not beeing in 3NF means that there exists an non-prime attribute A which is transitive dependent from a key X.
That mean that there exists Y s.t. $A \notin X$, $A \notin Y$ and we also have that: $X \to Y \in \Sigma^+$,$Y \to A \in \Sigma^+$ and that $Y \to X \notin \Sigma^+$.

Y does not contain any key (otherweise, we would have that anything can be determined from Y, including X so we would have that $Y \to X \in \Sigma^+$).

However, we have that $Y \to A \in \Sigma^+$ and also that (R, Σ) is in BCNF which mean that Y is a key (according to BCNF definition) and this contradicts the previous statment.
So, our supposition was false in the first place and (R, Σ) is indeed in 3NF.

Lossless join decompositions of schemas

Consider a relation schema $R[A_1, A_2, \ldots A_n]$. We say about a set $\rho = \{R_1[A_{i_1^1}, A_{i_2^1}, \ldots A_{i_{k_1}^1}], R_2[A_{i_1^2}, A_{i_2^2}, \ldots A_{i_{k_2}^2}] \ldots R_t[A_{i_1^t}, A_{i_2^t}, \ldots A_{i_{k_t}^t}]\}$ that it is a join decomposition of $R[A_1, A_2, \ldots A_n]$ if:

$$\bigcup_{p=1}^{t} \bigcup_{r=1}^{k_p} A_{i_r^p} = \{A_1 \ldots A_n\}$$

ρ is a lossless join decomposition with respect to a set of functional dependencies Σ if $\forall r$ over R that satisfy the set of functional dependencies Σ, we have that $r = r[A_{i_1^1}, A_{i_2^1}, ..A_{i_{k_1}^1}] \bowtie r[A_{i_1^2}, A_{i_2^2}, ..A_{i_{k_2}^2}] \bowtie .. \; r[A_{i_1^t}, A_{i_2^t}, ..A_{i_{k_t}^t}]$.

Lossless join decompositions of schemas

Theorem: If $\rho = \{R_1, R_2\}$ is a decomposition of R and Σ a set of functional dependencies, then ρ is a lossless join decomposition w.r.t. Σ iff $R_1 \cap R_2 \rightarrow R_1 - R_2 \in \Sigma^+$ or $R_1 \cap R_2 \rightarrow R_2 - R_1 \in \Sigma^+$ (the opreations are actually over the set of attributes on which the schemas were build)

Example: Consider $R[A, B, C]$ and $\Sigma = \{A \rightarrow B\}$.

$\rho_1 = \{R_1[A, B], R_2[A, C]\}$ is lossless because:
$AB \cap AC = A, AB - AC = B$ and $A \rightarrow B \in \Sigma^+$

$\rho_2 = \{R_1[A, B], R_2[B, C]\}$ is with loss because:
$AB \cap BC = B, AB - BC = A$ and $B \rightarrow A \notin \Sigma^+$
$AB \cap BC = B, BC - AB = C$ and $B \rightarrow C \notin \Sigma^+$
Test both decompositions for $r = \{(1, 1, 2), (1, 1, 3), (2, 1, 2)\}$.

Lossless join decompositions of schemas

We can compute Σ_i for $R_i[U_i]$ and continune the process of decomposition untill we get to relation schemas that are in BCNF. $\Sigma_i = \{X \to Y | X, Y \in U_i, X \to Y \in \Sigma^+\}$ - meaning that for $R_i[U_i]$ which is an element of the decomposition ρ we consider those dependencies from Σ^+ which are built over the attributes that are in U_i.

For the previous example:
Consider that $R[A, B, C]$ and $\Sigma = \{A \to B\}$.

$\rho_1 = \{R_1[A, B], R_2[A, C]\}$ is lossless join decomposition because:
$AB \cap AC = A, AB - AC = B$ si $A \to B \in \Sigma^+$

we have that $\Sigma_1 = \{A \to B\}$ and $\Sigma_2 = \emptyset$

Lossless join decomposition for BCNF algorithm

Input: (R, Σ)
Output: $\rho = \{(R_1, \Sigma_1), \ldots (R_t, \Sigma_t)\}$ = lossless decomposition of R w.r.t. Σ. Where (R_i, Σ_i) is in BCNF, $\forall i \in \{1..t\}$

Step 1: $\rho = R = R_1$; we find Σ^+ and the keys in R (which are required to check the BCNF).
Step 2: While $\exists (R_i, \Sigma_i) \in \rho$ and (R_i, Σ_i) is NOT in BCNF (it means that $\exists X \to A \in \Sigma_i$ and X is not a superkey in (R_i, Σ_i)):

- Step 2.1: Pick $X \to A$ from Σ_i s.t. $A \notin X$ and X does not contain any key.
- Step 2.2: $S_1 = X \cup \{A\}$; $\quad S_2 = R_i - A$;
- Step 2.3: $\rho = \rho - R_i$; $\quad \rho = \rho \cup S_1 \cup S_2$;
- Step 2.4: Compute $\Sigma^+_{S_1}, \Sigma^+_{S_2}$ and the keys for S_1 and S_2.

Example

Relational schema:
Absolvent(std_id, std_name, std_address, univ_id, univ_name, univ_town, rank, priority)
$\Sigma = \{$ std_id $\rightarrow$ std_name, std_address,rank rank $\rightarrow$ priority
univ_id $\rightarrow$ univ_name, univ_town$\}$

Will decompose in:
. . . let's compute. . .

Example

Relational schema:
Absolvent(std_id, std_name, std_address, univ_id, univ_name, univ_town, rank, priority)
$\Sigma =\{$ std_id $\rightarrow$ std_name, std_address,rank rank $\rightarrow$ priority
univ_id $\rightarrow$ univ_name, univ_town$\}$

Will decompose in:
$\rho = \{$ R1[univ_id, univ_name, univ_town], R2[rank, priority], R3[std_id, std_name, std_address, rank], R4[std_id, univ_id]$\}$

4NF (Ronald Fagin) (1977)

A relational schema R together with a set of multivalued dependencies Δ (delta) is in 4NF if it is in 1NF and for any non-trivial multivalued dependency $X \twoheadrightarrow A \in \Delta^+$, X is superkey in (R, Σ).

Observation: A relationa schema that is in 4NF is also in BCNF.

Proof by contradiction: suppose that (R, Σ, Δ) is not in BCNF. That mean that there exist a functional dependency $X \rightarrow A \in \Sigma^+$ and X is not a key. Then, because of the property of FD/MVD we have that any functional dependency is also a multivalued dependency and, therefore, there exists also the multivalued dependency $X \twoheadrightarrow A \in \Delta^+$ where X is not a key.
But that mean that R is not in 4NF (contradiction).

4NF lossless join decomposition algorithm

Input: (R, Σ, Δ)
Output: $\rho = \{R_1, \ldots, R_t\}$ = lossless join decomposition of R w.r.t Σ. Where $(R_i, \Sigma_i, \Delta_i)$ is in 4NF, $\forall i \in \{1..t\}$

Step 1: $\rho = R = R_1$; we compute Σ^+, Δ^+ and the candidate keys in R (in order to check the 4NF).
Step 2: While $\exists (R_i, \Sigma_i, \Delta_i) \in \rho$ that is not in 4NF (meaning that $\exists X \twoheadrightarrow A \in \Delta^+$ and X is not a superkey):

Step 2.1: Choose $X \twoheadrightarrow A$ from Σ_i s.t. $A \notin X$ and X is not a superkey
Step 2.2: $S_1 = X \cup \{A\}; \quad S_2 = R_i - A;$
Step 2.3: $\rho = \rho - R_i; \quad \rho = \rho \cup S_1 \cup S_2;$
Step 2.4: Compute $\Sigma^+_{S_1}, \Delta^+_{S_1}, \Sigma^+_{S_2}, \Delta^+_{S_2}$ and the keys for S_1 and S_2.

4NF decomposition example

Consider the following schema:
student[social_number, name, faculty, passion]

The student can be a student at multiple faculties and have more then one passion:
social_number $\rightarrow$ name;
social_number,name $\twoheadrightarrow$ faculty;

Will decompose in:
$\rho = \{$S1[social_number, name], S2[social_number, faculty], S3[social_number, passion] $\}$.

Bibliografie

- Further Normalization of the Data Base Relational Model. - *Frank Edgar Codd*; IBM Research Report RJ909 (August, 1971)
- Baze de date relaționale. Dependențe - *Victor Felea*; Univ. Al. I. Cuza, 1996